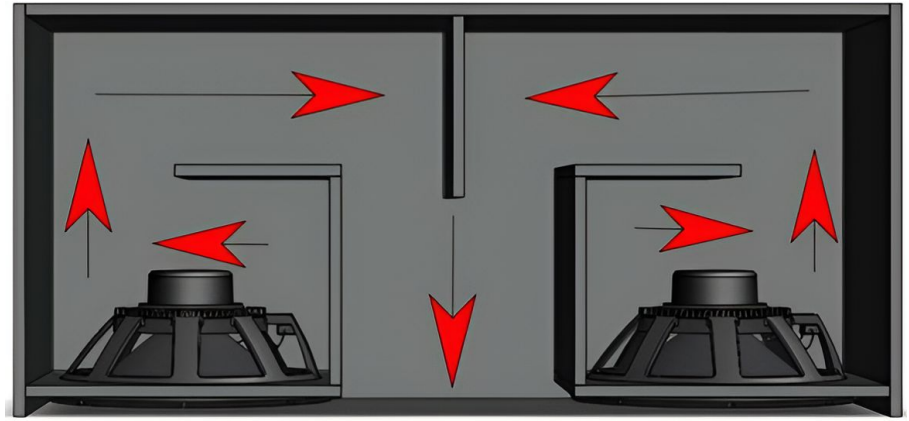
Optimization Methods for Engineer:



Subwoofer Transmission Line Optimization

Joachim Zivy (19-923 093), Julio Schneider (22-945-828)

Image: https://soundagency.fr/docs/les-charges-acoustiques/

Goal:

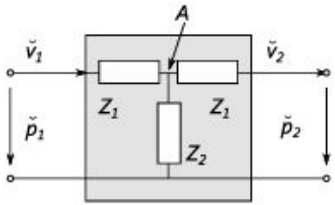
- Given a book of specifications, find the optimal transmission line design that matches or approaches the required performances under some constraints
- Automate the generation of ABEC3 simulation scripts

Motivation:

- Speaker development is a trial and error process
- Simulations can be extensively long depending on the simulation method
- Complexity of search space

Definition

- A transmission line is an acoustic waveguide made of a series of acoustic ducts.
- A duct D= (H, W, L) is an acoustic waveguide of constant cross-section HxW and of length L. It's electrical equivalent network can be found below.
- An acoustic mass is accelerated but uncompressed air and is used to model the impedance discontinuity at the interface of 2 different cross section ducts. It's electrical analog is an inductance. $Z = j\omega\frac{\rho l}{A}$

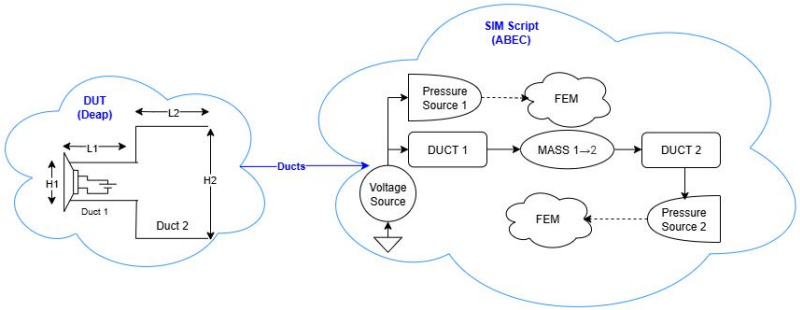


$$Z_1 = j\rho c\frac{1-\cos(kd)}{\sin(kd)}$$

$$Z_2 = \frac{-j\rho c}{\sin(kd)}$$

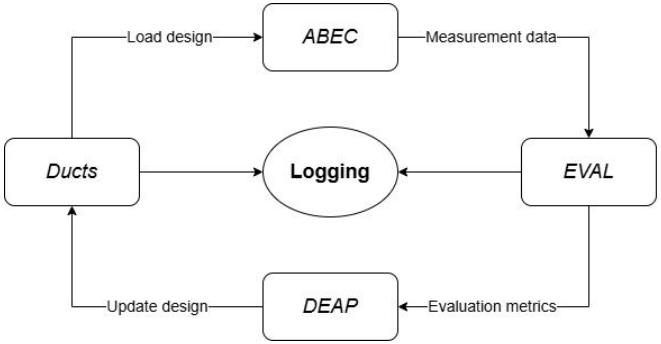
Source: Kurt Heutschi, Lecture notes on Acoustic 2, 2020-08-07

Tools:



- Deap: Python based Genetic algorithm library
- ABEC3: LEM & FEM acoustic simulation software
- PyGetW: Python based library for GUI I/O
- Python script to generate ABEC3 model as well as the evaluation of the measurement data

Pipeline:



- Generate a random design and load into ABEC3
- Simulate the TL and the resulting pressure source using Lumped Element Method.
- Measurements are done using the Finite Element Method.
- Evaluate measurement data and feed metric to DEAP

ABEC3 (AKABAK3): https://www.randteam.de/AKABAK3/

Algorithm:

Deap is a python library for the implementation of genetic algorithm.

1. Generate P random TL design {p}

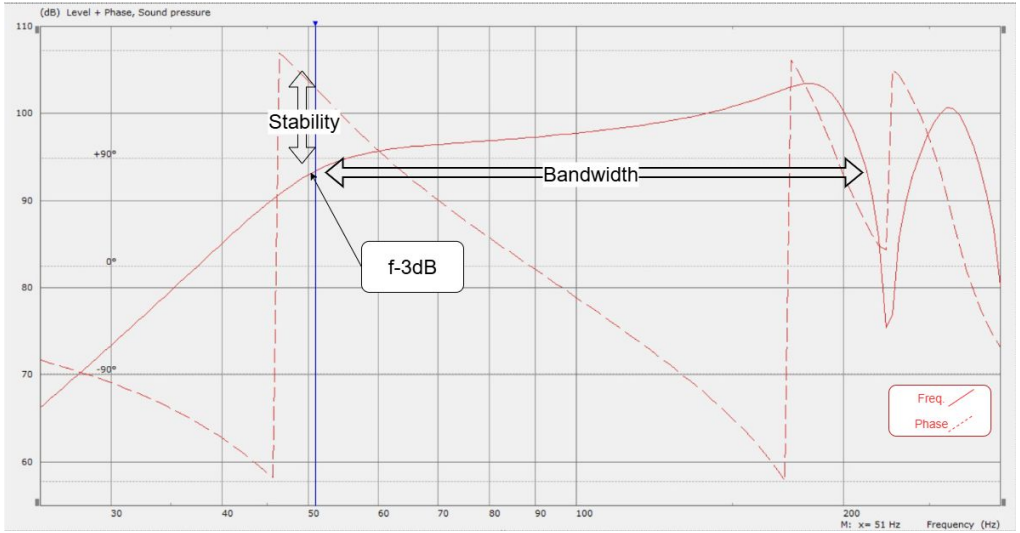
2. Simulate in ABEC
3. Evaluate the SPL and phase response and attribute fitness
4. Select t random design {s}
5. Choose the k best design and propagate to the next generation.
6. Mute or exchange ducts among the best designs

7. If g = gen: exit
8. Increment g.
9. Go back to 2.

DEAP: https://github.com/deap/deap

Evaluation:

- F-3dB (Lower frequency extension)
- Lower and upper limiting frequency for analysis
- Tolerance (Maximum phase variations)
- Transmission line Width (W kept constant across all ducts)
- Maximum transmission line length
- Maximum amount of ducts



Lexicon:

- Bw = BandWidth
- Fr = Frequency Response
- Ph = Phase Response
- Gd = Group Delay = d/df(Ph)
- Std = STandard Deviation

Implementation: *Deap parameters*

Variable	Pop	Gen	Tourn_size	k	Mut_prob	Cross_prob	rem_duct	add_duct	Modif_legth	Mofif_width
Formula	p	g	t	k	MUBP	CXBP	rem_duct	add_duct	mod_len	mod_wid
Aim	Size of population	Number of generations	Number of randomly selected individual to compete	Number of winners	Probability of mutation	Probability of crossover	Probability of removing a duct	Probability of adding a duct	Probability of modifying the length if a duct	Probability of modifying the W of a duct

Fitness functions:

Variable	n_spl	n_std	n_dev	n_range	cut_off	s_phase	n_gstd	n_maxgd
Formula	mean(Fr_Bw)	σ(Fr_Bw)	Max(Abs(fr_bw-n_spl)	cutoff_h-cutoff_l	fr[n_spl-3]	Phase within	σ(- ▽ (Phase))	Max(n_gstd)
Aim	Max sensitivity	Min std	Min outliers	Max bandwidth	Min low frequency	Max stable phase	Min GD	Min GD spikes

Testing

The test we're done with the following parameters:

- P: 30
- G: 15
- Mut_prop: 0.3
- Cros_prob: 0.8
- Add_duct = Rem_duct = Mod_len = Mod_width = 0.25
- Toun_size: p/2
- k=3
- (elites = 1)
- random.seed=1

- With elitism, the algorithm seemed to stop evolution around 5-10 generations if the beginning population was big enough. P=30 and G=15 seemed to be enough

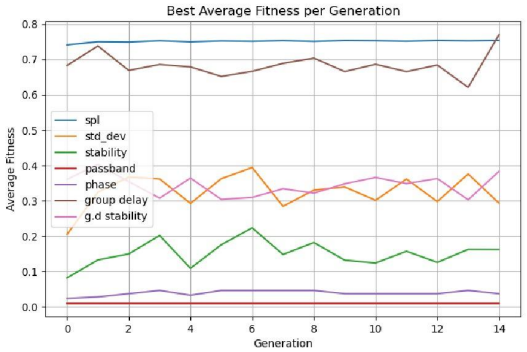
- The tourn_size was set in order to have a good probability of selecting the best individuals, while still taking weaker path in order to explore to large region of designs.

- K = 3 seemed to search the result space the longest.

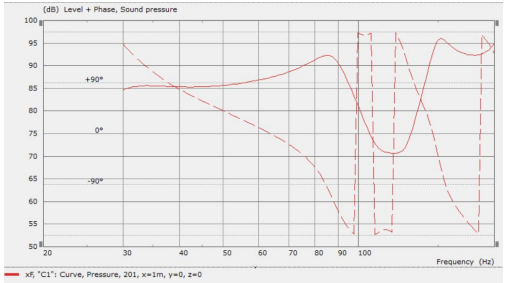
- Having elites=1 (best individual of a generation) allowed for the best design genetics to be copied to the next generation. This ensured this design would survive until the last generation.

Results without elitism:

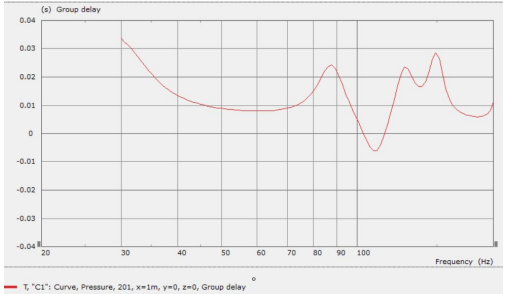
The algorithm is forced to alter the best individuals. This has for consequences that the best individual doesn't necessarily survives until the last generations



The resulting SPL curves curves are descent. The passband region 30-90 Hz is quite flat and the sensitivity is good for the driver. The phase response is steady which helps with phase alignment in crossover regions.

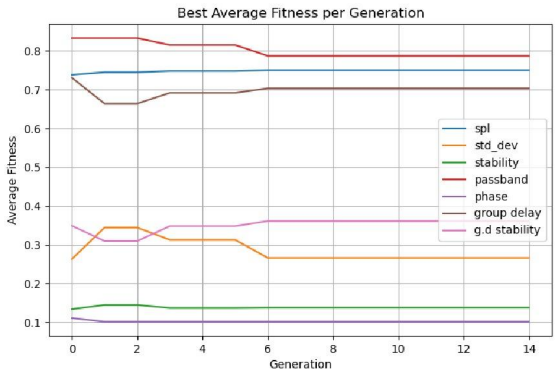


Group delay behaves quite well. The 30 ms threshold is never exceeded, which is an ideal range for low frequency applications.

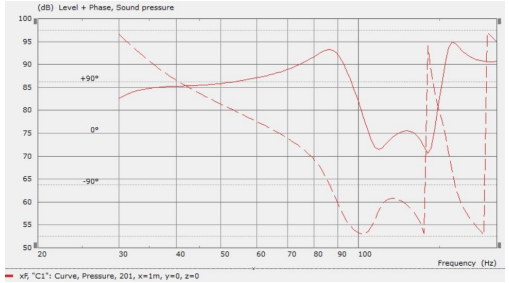


Results with elitism:

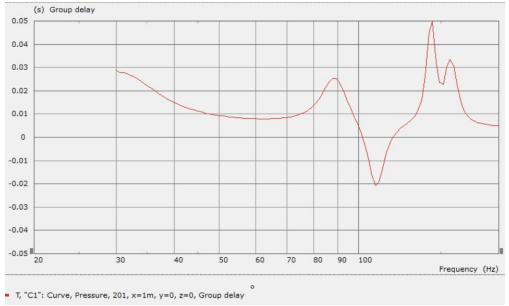
The individual with the best genetics (fitness evaluations) is always copied to the next generations. This ensure to preserve the best genetics found among all generations.



The SPL curve is quite similar to the SPL curve without elitism. It could mean that the algorithm without elitism “gravitates” around the ideal solution it encountered in previous generations.



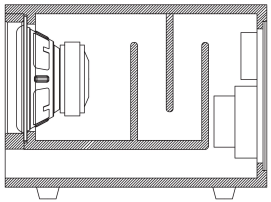
Group delay behaves quite well. The 30 ms threshold is never exceeded, which is an ideal range for low frequency applications.



Conclusions:

- The search space is complex. It depends on a lot of factor and some fitness evaluations criteria are dependant on another. As a result the tool converges towards a compromise between the different criterias.
- By changing the weights of the different fitness values the user can influence the tool into prioritizing certain genetics.
- Although the responses differ, they seem to follow a similar tendency. This could come from random generator seed. It could also mean our implementation isn’t spanning a wide enough search space.
- The tool produces good results and could help in accelerating r&d process when prototyping subwoofer designs by constraining the tool according to the architecture needed.

Improvements



- Other critical criterias of subwoofer are their power capacity. This strongly impact capable RMS SPL and is limited by factors as driver excursion or air flow velocity. Our algorithm doesn’t consider those characteristics to rate a design,
- The tool is working by manually opening the simulation software and using virtual key presses to activate shortcuts to navigate the simulations software. This resulted in ~ 15 min run time for 300 generations.This could get problematic if the need arises to simulate a variety of design after another.
- BEM simulations are usually more accurate. And adaptation would be possible but would involve geometry and mesh generations, as well as a lot more intensive solving stage in terms of CPU usage.